DEVSECOPS LAB BOOK · PHASE 6: CONTAINER ORCHESTRATION (ADVANCED)

Chapters 25–26

Istio & GitOps

Service Mesh, Traffic Management & Automated Deployments

Chapter 25

Istio Service Mesh

Chapter 26

GitOps with ArgoCD

CHAPTER 25 · GOAL

Deploy Istio Service Mesh

Advanced traffic management, security, and observability across every service — without changing application code.

Traffic Management

Canary rollouts, weighted routing, retries, and timeouts.

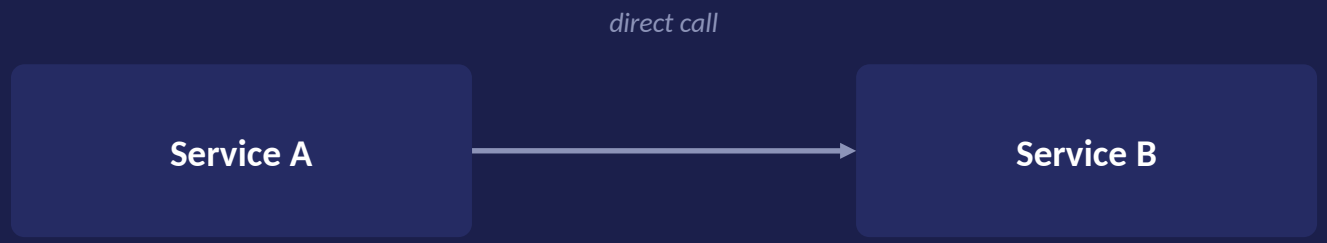
Zero-Trust Security

Automatic mTLS between every service, transparently.

Deep Observability

Prometheus metrics and Kiali topology, out of the box.

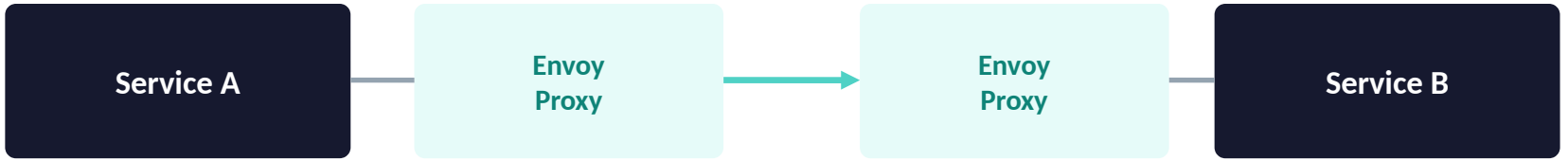
Before Istio — Logic Lives in the App



Every one of these has to be built, tested, and maintained inside the application:

- ✖ Retry logic in application code
- ✖ Timeouts configured in app
- ✖ Load balancing in app
- ✖ Security (mTLS) in app
- ✖ Circuit breakers in app
- ✖ Observability scattered

With Istio — the Mesh Handles It



The mesh transparently handles all cross-cutting concerns:

- Retries (automatic)
- Timeouts (automatic)
- Load balancing (sophisticated)
- mTLS (automatic encryption)
- Circuit breakers (automatic)
- Observability (Prometheus + Grafana)

Applications remain simple.

Install & Enable the Mesh

```
# Download Istio
curl -L https://istio.io/downloadIstio | sh -
cd istio-1.15.0
export PATH=$PWD/bin:$PATH

# Install Istio
istioctl install --set profile=demo -y

# Label namespace for injection
kubectl label namespace default \
  istio-injection=enabled

# Verify installation
kubectl get all -n istio-system
```



Download the CLI

Fetch istioctl and add it to PATH.



Install the control plane

The demo profile installs Istiod + gateways.



Enable sidecar injection

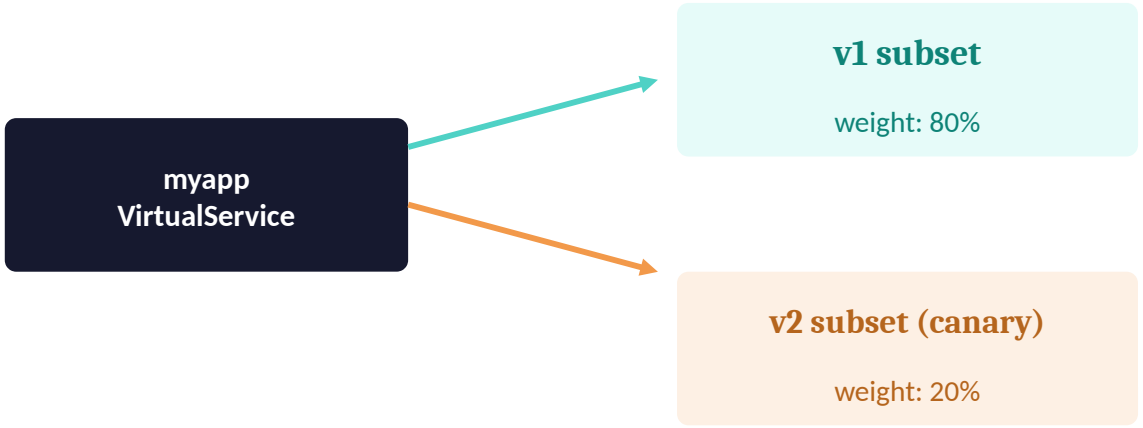
Label the namespace — pods get an Envoy sidecar automatically.



Verify

Confirm istio-system pods are running.

Weighted Routing — Canary Deployments



timeout: 10s · retries: 3 attempts, perTryTimeout 2s

Canary rollout

1. Deploy v1 (current) and v2 (canary)
2. Apply VirtualService — 80% v1 / 20% v2
3. Watch v2 error rate & latency in Prometheus
4. Ramp traffic: 20% → 50% → 100%

Mutual TLS — Encrypted by Default

```yaml
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
spec:
  mtls:
    mode: STRICT   # enforce mTLS for all traffic
```



STRICT MODE

No plaintext traffic is permitted between mesh services.

- All service-to-service communication encrypted
- Mutual authentication — each service verifies the other
- Certificate management automatic (Istio handles rotation)
- Transparent to applications

Metrics, Latency & Topology

Prometheus Metrics

Istio automatically exports request rate, error rate, and latency percentiles (p50 / p90 / p99) for every service.

```
rate(istio_requests_total[5m])

histogram_quantile(0.99,
  rate(istio_request_duration_
  milliseconds_bucket[5m]))
```

Kiali Visualization

Port-forward and open the dashboard to see the live mesh topology, traffic flow, and error rates.

```
kubectl port-forward -n istio-system \
  svc/kiali 20000:20000
```

Visit http://localhost:20000 to see:

- Service mesh topology
- Live traffic flow
- Error rates per edge



CHAPTER 26

GitOps with ArgoCD

Git as the source of truth. Every change to Kubernetes flows through a commit.

The GitOps Flow

1

Developer commits code to Git

2

CI pipeline tests & builds (Phase 4)

3

Push image to Harbor

4

Update Kubernetes manifests in Git repo

5

ArgoCD detects the Git change

6

Automatically deploys to Kubernetes

Desired state (Git) = Actual state (Kubernetes)

Install & Access ArgoCD

```
# Create namespace
kubectl create namespace argocd

# Install ArgoCD
kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/\
  argo-cd/stable/manifests/install.yaml

# Access ArgoCD UI
kubectl port-forward -n argocd \
  svc/argocd-server 8080:443

# Get initial password
kubectl -n argocd get secret \
  argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d
```

Login

Visit https://localhost:8080

admin / <initial password>



ArgoCD continuously reconciles cluster state to match Git.

Application Manifest & Sync Policy

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: myapp
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://gitlab.devsecops.com/
      root/myapp-helm.git
    targetRevision: main
    path: helm/myapp
  destination:
    server: https://kubernetes.default.svc
    namespace: myapp
  syncPolicy:
    automated:
      prune: true      # delete if removed from Git
      selfHeal: true   # resync if drift detected
    syncOptions:
    - CreateNamespace=true
```

Sync policy in plain terms

`prune: true`

Resources removed from Git are removed from the cluster too.

`selfHeal: true`

Manual cluster drift is automatically reverted to match Git.

`CreateNamespace`

ArgoCD creates the target namespace if it doesn't exist.

```
kubectl apply -f argocd-application.yaml
```

One Repo, Every Environment

```
myapp-helm/
├── helm/
│   ├── myapp/
│   │   ├── Chart.yaml
│   │   ├── values.yaml
│   │   ├── values-dev.yaml
│   │   ├── values-prod.yaml
│   │   └── templates/
│   └── postgresql/
└── argocd/
    ├── apps/
    │   ├── myapp-dev.yaml
    │   ├── myapp-prod.yaml
    │   └── postgresql.yaml
    └── projects/
        └── default.yaml
```



`helm/`

The Helm chart(s) that define how the app is deployed — one folder per app.



`values-*.yaml`

Per-environment overrides — dev gets 1 replica, prod gets pinned versions.



`argocd/apps/`

One ArgoCD Application manifest per environment, pointing at this same repo.

26.5 COMPLETE GITOPS WORKFLOW

Shipping a New Version — No kubectl Apply

1	**Push image**	CI pipeline builds & pushes new image to Harbor, tags the release.
2	**Edit values.yaml**	Change tag: "1.0.0" → tag: "1.1.0" in the Helm chart.
3	**Commit & push**	git commit -m "Update myapp to v1.1.0" && git push origin main
4	**ArgoCD auto-syncs**	ArgoCD detects the Git change and deploys automatically.
5	**Verify**	kubectl get pods -n myapp shows the new version running.

No manual kubectl apply needed — Git is the only interface to the cluster.

Same Chart, Different Values

myapp-dev.yaml

```
source:
  path: helm/myapp
  helm:
    values: |
      replicaCount: 1
      image:
        tag: latest
destination:
  namespace: myapp-dev
```

myapp-prod.yaml

```
source:
  path: helm/myapp
  helm:
    values: |
      replicaCount: 5
      image:
        tag: "1.1.0"  # pinned
destination:
  namespace: myapp-prod
```

GitLab CI → ArgoCD

```
# .gitlab-ci.yml — update Helm values after push
deploy:
  stage: deploy
  script:
    - git clone https://gitlab.devsecops.com/
        root/myapp-helm.git
    - cd myapp-helm
    - sed -i "s/tag: .*/tag: \"$CI_COMMIT_SHA\"/" \
        helm/myapp/values.yaml
    - git config user.email "ci@devsecops.com"
    - git config user.name "CI Bot"
    - git add helm/myapp/values.yaml
    - git commit -m "Update image to $CI_COMMIT_SHA"
    - git push https://oauth2:$GITLAB_TOKEN@...
        main   # ArgoCD syncs automatically
```

Why this matters

- CI never touches the cluster directly
- Every deploy is a Git commit — full audit trail
- ArgoCD's selfHeal + prune keep cluster in sync

Undo a Bad Deploy in One Command

Automatic Rollback

ArgoCD can automatically roll back if health checks fail after a sync.

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
    # auto-rollback on
    # failed sync
```

Manual Rollback

Step back to any previous sync from the CLI.

```
# Check sync history
argocd app history myapp

# Rollback to previous sync
argocd app rollback myapp 2

# Result: Git is reverted,
# ArgoCD deploys old version
```

The Toolchain, End to End

Tool	Purpose
Istio	Service mesh — traffic, security, observability
ArgoCD	GitOps deployment automation
Git	Source of truth
Kubernetes	Actual deployed state

KEY TAKEAWAY

Git-Driven Infrastructure

BEFORE GITOPS

- ✗ Manual kubectl apply
- ✗ Drift between Git and cluster
- ✗ Hard to track changes
- ✗ Difficult rollbacks

AFTER GITOPS (ARGOCD)

- ✓ All changes through Git commits
- ✓ Git = source of truth
- ✓ Auto-sync to cluster
- ✓ Easy rollbacks (revert commit)
- ✓ Audit trail in Git